

Case 1: Booking List (filter + sort + pagination)

1. Tình huống thực tế:

Bạn đang làm trang admin để hiển thị danh sách booking của hệ thống đặt phòng khách sạn. Trang này có:

- Lọc theo trạng thái (status) ví dụ CONFIRMED
- Lọc theo khoảng thời gian tạo (created_at)
- Sắp xếp mới nhất trước (mới → cũ)
- Phân trang (pagination)

Bảng liên quan: bookings, employees, hotels

Index hiện có: SQLCREATE INDEX idx_bookings_status_created ON bookings(status, created_at DESC) → Index này giúp PostgreSQL có thể vừa filter theo status, vừa sort theo created_at DESC ngay trên index.

2. Câu query hiện tại (rất phổ biến nhưng chậm):

```
SELECT b.*, e.name, h.name
FROM bookings b
JOIN employees e ON b.employee_id = e.id
JOIN hotels h ON b.hotel_id = h.id
WHERE b.status = 'CONFIRMED'
      AND DATE(b.created_at) BETWEEN '2024-01-01' AND '2024-01-31'
ORDER BY b.created_at DESC
LIMIT 20 OFFSET 10000;
```

3. Nguyên nhân 1: DATE(created_at) phá hủy index

Khi bạn dùng hàm DATE() hoặc bất kỳ hàm nào bọc quanh cột created_at → PostgreSQL không thể dùng index idx_bookings_status_created nữa.

→ Kết quả: database phải quét toàn bộ bảng (Sequential Scan) → chậm kinh khủng khi bảng có hàng triệu dòng.

Giải quyết nguyên nhân 1:

Viết lại điều kiện thời gian theo dạng phạm vi trực tiếp (range) để tận dụng index:

```
WHERE b.status = 'CONFIRMED'
      AND b.created_at >= '2024-01-01 00:00:00'
      AND b.created_at < '2024-02-01 00:00:00'
```

→ Index trên (status, created_at DESC) sẽ được dùng ngay lập tức.

4. Nguyên nhân 2: OFFSET lớn cực kỳ tốn kém

Khi bạn dùng OFFSET 10000 LIMIT 20, PostgreSQL vẫn phải:

- Tìm và sắp xếp 10.020 dòng đầu tiên
- Sau đó bỏ đi 10.000 dòng

→ Càng lùi sâu (page 500, 1000...) thì càng chậm dần, thậm chí vài giây đến vài chục giây.

Giải quyết nguyên nhân 2:

Chuyển sang cursor-based pagination (phân trang theo con trỏ / keyset pagination).

Thay vì OFFSET, ta truyền giá trị `created_at` của dòng cuối cùng ở trang trước, rồi lấy các dòng nhỏ hơn nó.

Ví dụ:

Trang đầu: không truyền `last_created_at`

Trang sau: truyền `last_created_at = '2024-01-15 14:30:22'`

5. Nguyên nhân 3: JOIN trước rồi mới LIMIT → lãng phí

Bạn join hàng chục nghìn dòng bookings với employees và hotels, sau đó mới LIMIT/OFFSET → tốn CPU, tốn RAM vô ích.

Giải quyết nguyên nhân 3:

Dùng late join (join muộn):

- Lấy trước chỉ 20 id booking thỏa mãn filter + sort (rất nhẹ)

- Sau đó mới join với employees và hotels (chỉ join 20 dòng)

Câu query sau khi optimize (phiên bản khuyến nghị tốt nhất):

```
SELECT
    b.id,
    b.created_at,
    b.total_price,
    e.name AS employee_name,
    h.name AS hotel_name
FROM (
    SELECT id, employee_id, hotel_id, created_at
    FROM bookings
    WHERE status = 'CONFIRMED'
        AND created_at >= '2024-01-01 00:00:00'
        AND created_at < '2024-02-01 00:00:00'
        AND created_at < :last_created_at      -- cho trang sau (cursor)
    ORDER BY created_at DESC
    LIMIT 20
) b
JOIN employees e ON b.employee_id = e.id
JOIN hotels h ON b.hotel_id = h.id
ORDER BY b.created_at DESC;      -- giữ thứ tự
```

6. Tổng kết – Tôi đã làm những việc sau để giúp query nhanh hơn:

a → Viết lại điều kiện thời gian (không dùng DATE()) → tận dụng được index → tránh seq scan toàn bảng

b → Tạo composite index (status, created_at DESC) → filter + sort cùng lúc đều dùng index → cực nhanh
c → Chuyển từ OFFSET sang cursor-based pagination (dùng created_at làm con trỏ) → không cần scan lại dữ liệu cũ → page 500 vẫn nhanh như page 1 *chỉ dùng cho infinite scroll (cuộn xuống load thêm), không dùng cho tính năng nhảy page
d → Dùng late join (subquery lấy booking trước, join sau) → giảm số lượng join xuống chỉ còn 20 dòng → tiết kiệm CPU + memory rất nhiều
Khi kết hợp cả 4 thay đổi trên → query thường nhanh hơn 10–100 lần (đặc biệt khi đi sâu vào các trang sau). Đây là cách làm chuẩn cho các hệ thống có lượng booking lớn ở Việt Nam hiện nay.

Case 2: Check duplicate booking (time overlap)

1. Tình huống thực tế:

Trước khi cho phép tạo một booking mới cho nhân viên (employee), hệ thống cần kiểm tra xem nhân viên đó đã có booking nào trùng thời gian (check-in và check-out chồng lấn) chưa.

Nếu có → không cho tạo, báo lỗi "Nhân viên đã có lịch trong khoảng thời gian này".

Bảng liên quan: bookings (chứa employee_id, checkin_date, checkout_date, status, ...)

Hiện tại chưa có index nào hỗ trợ query overlap.

2. Câu query hiện tại (phổ biến nhưng chậm khi dữ liệu lớn):

```
SELECT *  
FROM bookings  
WHERE employee_id = 123  
      AND status IN ('CONFIRMED', 'PENDING')  
      AND checkin_date <= '2024-02-10'  
      AND checkout_date >= '2024-02-05';
```

Query này kiểm tra xem có booking nào của employee 123 mà khoảng thời gian chồng lấn với booking mới (từ 2024-02-05 đến 2024-02-10) hay không.

3. Nguyên nhân 1: Điều kiện overlap rất khó dùng index thông thường (B-tree)

- PostgreSQL không thể dùng index trên checkin_date và checkout_date riêng lẻ cho điều kiện checkin <= X AND checkout >= Y.

- Không có index nào hỗ trợ tốt kiểu "hai cột phải thỏa mãn cùng lúc theo quan hệ chồng lấn".

→ Kết quả: database thường phải Sequential Scan (quét toàn bộ bảng bookings) hoặc quét rất nhiều row → chậm khi bảng có hàng trăm nghìn / triệu booking.

Giải quyết nguyên nhân 1:

Sử dụng kiểu dữ liệu range có sẵn trong PostgreSQL: tsrange (hoặc daterange nếu chỉ cần ngày).

PostgreSQL hỗ trợ toán tử && (overlap) → kiểm tra hai khoảng thời gian có giao nhau không.

→ Query trở nên rõ ràng và tự nhiên hơn rất nhiều:

```
tsrange(checkin_date, checkout_date) && tsrange('2024-02-05', '2024-02-10')
```

4. Nguyên nhân 2: Không có index phù hợp → scan toàn bộ hoặc rất nhiều row

Dù bạn có index riêng trên employee_id, hoặc trên checkin_date/checkout_date,

PostgreSQL vẫn không thể dùng chúng hiệu quả cho điều kiện overlap → vẫn phải đọc rất nhiều dòng để lọc.

Giải quyết nguyên nhân 2:

Tạo GiST index (Generalized Search Tree) – loại index chuyên dụng cho dữ liệu không gian, khoảng thời gian, hình học...

GiST index hỗ trợ cực tốt các toán tử như && (overlap), @> (chứa), <@ (nằm trong), v.v.

```
CREATE INDEX idx_booking_employee_range
ON bookings
USING GIST (
    employee_id,
    tsrange(checkin_date, checkout_date)
);
```

→ Index này cho phép PostgreSQL nhảy thẳng đến các booking của employee_id cụ thể và kiểm tra overlap rất nhanh bằng cách dùng cấu trúc cây không gian.

5. Nguyên nhân 3: Query lấy toàn bộ cột (*) và không dùng sớm

- SELECT * → tốn tài nguyên đọc dữ liệu không cần thiết.

- Không có LIMIT → database vẫn tiếp tục tìm dù chỉ cần biết "có ít nhất 1 booking trùng hay không".

Giải quyết nguyên nhân 3:

- Chỉ cần kiểm tra tồn tại → dùng SELECT 1 hoặc EXISTS.

- Thêm LIMIT 1 để PostgreSQL dừng ngay khi tìm thấy booking đầu tiên trùng.

Câu query sau khi optimize (phiên bản tốt nhất):

```
SELECT 1
FROM bookings
WHERE employee_id = 123
    AND status IN ('CONFIRMED', 'PENDING')
    AND tsrange(checkin_date, checkout_date, '[]') -- [] để xử lý đúng
    biên (tùy business rule)
```

```

        && tsrange('2024-02-05', '2024-02-10', '[]')
LIMIT 1;

```

Hoặc dùng EXISTS (thường sạch và dễ đọc hơn):

```

SELECT EXISTS (
    SELECT 1
    FROM bookings
    WHERE employee_id = 123
        AND status IN ('CONFIRMED', 'PENDING')
        AND tsrange(checkin_date, checkout_date, '[]')
            && tsrange('2024-02-05', '2024-02-10', '[]')
LIMIT 1
);

```

6. Tổng kết – Tôi đã làm những việc sau để giúp query nhanh hơn rất nhiều:

a → Chuyển điều kiện overlap sang dùng tsrange + toán tử && → query rõ ràng, tự nhiên, và PostgreSQL hiểu ngay đây là overlap range

b → Tạo GiST index trên (employee_id, tsrange(checkin_date, checkout_date)) → database không còn quét toàn bảng, mà dùng index để tìm nhanh các khoảng thời gian chồng lấn của đúng employee

c → Chỉ SELECT 1 + LIMIT 1 (hoặc EXISTS) → dừng ngay khi tìm thấy booking trùng đầu tiên → tiết kiệm I/O, CPU, thời gian

Khi kết hợp cả 3 thay đổi này, query kiểm tra trùng lịch thường chỉ mất vài millisecond ngay cả khi bảng bookings có hàng triệu dòng – thay vì vài giây hoặc timeout như query cũ. Đây là cách làm chuẩn cho các hệ thống đặt lịch, đặt phòng, lịch làm việc ở Việt Nam hiện nay (khách sạn, spa, bác sĩ, giáo viên...).

Case 3: Report theo ngày (aggregation)

1. Tình huống:

Trong hệ thống dashboard, bạn cần hiển thị:

- Số lượng booking mỗi ngày
- Tổng doanh thu mỗi ngày

Dữ liệu chính nằm ở bảng: bookings (có created_at, price)

Hiện tại: chưa có index gì cả

2. Query ban đầu

```

SELECT DATE(created_at) as date,
       COUNT(*) as total,
       SUM(price) as revenue
FROM bookings
WHERE created_at >= '2024-01-01'
GROUP BY DATE(created_at)

```

`ORDER BY date;`

3. Nguyên nhân 1: dùng `DATE(created_at)` phá index

Điều này khiến:

- PostgreSQL phải tính `DATE()` cho từng row
 - Không thể dùng index trên `created_at`
- Kết quả: full table scan và gây chậm khi dữ liệu lớn

Giải quyết nguyên nhân 1: Expression Index

Tạo index trực tiếp trên expression:

```
CREATE INDEX idx_booking_created_date ON bookings  
((DATE(created_at)));
```

PostgreSQL cho phép index trên kết quả của function

Kết quả: query vẫn giữ nguyên nhưng đã dùng được index

4. Nguyên nhân 2: GROUP BY trên bảng lớn `GROUP BY DATE(created_at)`

Khi dữ liệu lớn, DB phải scan toàn bộ bảng, sau đó group + sort

→ rất tốn: CPU, memory, thời gian

Giải quyết nguyên nhân 2: Materialized View

Thay vì tính toán mỗi lần query → precompute trước

```
CREATE MATERIALIZED VIEW daily_stats AS  
SELECT DATE(created_at) as date,  
       COUNT(*) as total,  
       SUM(price) as revenue  
FROM bookings  
GROUP BY DATE(created_at);
```

Khi cần update: `REFRESH MATERIALIZED VIEW daily_stats;`

Ý tưởng: tính toán sẵn và lưu kết quả lại như 1 bảng

5. Query sau khi optimize

Nếu dùng expression index:

```
SELECT DATE(created_at) as date,  
       COUNT(*) as total,  
       SUM(price) as revenue  
FROM bookings  
WHERE DATE(created_at) >= '2024-01-01'  
GROUP BY DATE(created_at)  
ORDER BY date;
```

→ Nhanh hơn do dùng index

Nếu dùng materialized view (best practice):

```
SELECT *  
FROM daily_stats  
WHERE date >= '2024-01-01';
```

Gần như: không cần scan bảng lớn, tốc độ rất nhanh (gần $O(1)$)

6. Tổng kết:

Tôi đã tạo expression index (a) → giúp query dùng được index dù có DATE() (a1)

Tôi đã tối ưu aggregation bằng materialized view (b) → tránh phải GROUP BY trên bảng lớn mỗi lần (b1)

Tôi đã tách compute và query (c) → giảm tải runtime, tăng tốc dashboard (c1)

Kết quả:

Câu truy vấn trở nên nhanh hơn rất nhiều, đặc biệt khi dữ liệu lớn và dashboard được gọi thường xuyên

Giải thích nội dung trên thật dễ hiểu theo đoạn văn bản:

Tình huống:

Bảng trong database thực tế: bookings, employees, hotels

Chưa có index gì cả

Câu query hiện tại.

Nguyên nhân 1:

Giải quyết nguyên nhân 1:

Nguyên nhân 2:

Giải quyết nguyên nhân 2: ...

Câu query sau khi optimize:

Tổng kết: tôi đã làm a để a1, b để b1, c để c1 giúp cho câu truy vấn trở nên tốt hơn